

Testathon 2026

Comprehensive Testing Report

SKILLAB Platform

Justification, Methodology, and Findings

Submitted to:

SKILLAB Project Evaluation Committee

Prepared by:

Team DEDSEC

Staver Maxim, Florin-Marian Pușcaș, Batin Cătălin-Ioan

Cluj-Napoca, Romania

May 2026

Contents

1	Executive Summary	2
2	Methodology	2
2.1	Phase 1: Environment Setup & Automated E2E Testing	2
2.2	Phase 2: Manual Exploratory & Security Testing	3
3	Key Findings	3
3.1	Critical Identity, Authorization, and Security Failures	4
3.2	Role-Based Access Control (RBAC) and Routing Collapse	4
3.3	System Stability, API Integrity, and UI Inconsistencies	4
4	Complete Defect Tracker	5
4.1	Deliverables Included in the Testing Package	5
4.2	How to Run the Tests	5
5	Recommendations	6
5.1	Enforce Zero-Trust API Architecture and Strict Authorization	6
5.2	Repair Enterprise Routing Middleware and Session Management	7
5.3	Implement Global Exception Handling and Rate Limiting	7
5.4	Sanitize Inputs and Stabilize External Integrations	7
	Annex 1: Security and QA Findings	9

1 Executive Summary

Over the course of the one-month SKILLAB Testathon, our team conducted a comprehensive testing campaign on the open-source SKILLAB platform. Our goal was to evaluate the system's overall health, focusing on functional correctness, user interface stability, security boundaries, and load performance.

We identified three major areas of concern that need immediate attention before a production release:

- **Authorization & Data Leaks:** The backend API (specifically the `/user-management/user` endpoint) does not properly verify who is requesting data. This allowed us to easily view other users' personal information, password hashes, and reset codes. Because of this, anyone can completely take over another user's account without needing their password.
- **Broken Dashboards for Enterprise Roles:** The Role-Based Access Control (RBAC) is currently blocking key stakeholders. When Admin, Industry, or Policy users log in, the platform immediately redirects them to the public homepage or login screen, completely locking them out of their workspaces.
- **API Stability and Error Handling:** The system struggles with unexpected inputs. Submitting invalid data or bypassing the frontend often causes the backend to crash and reveal sensitive internal server information. Additionally, critical endpoints like the Job Sources API frequently time out and freeze, breaking core features.

Because the authorization issues and RBAC lockouts directly break the core purpose of the platform, we highly recommend fixing the API permission checks and the enterprise routing logic before adding any new features.

2 Methodology

Because we had a limited timeframe to evaluate a large platform, we decided against relying on just one type of testing. Instead, we divided our team's efforts into two distinct streams: automated End-to-End (E2E) testing to cover the core architecture repeatedly, and manual exploratory testing to find security flaws and edge cases that scripts often miss.

2.1 Phase 1: Environment Setup & Automated E2E Testing

- **Deployment Testing:** Before writing any tests, we verified the release build by deploying the platform locally using the provided Docker containers. We tested the initialization of the frontend, backend, and database to ensure the environment was stable for our test suite.
- **Functional & Integration Testing:** We used Playwright to build automated End-to-End (E2E) scripts. We created separate browser contexts to programmatically log in as all five user roles. This automated functional testing is what consistently revealed that the integration between the frontend routing and the backend authorization logic was failing for Industry, Policy, and Admin users, effectively locking them out.

- **API & Integration Reliability:** Instead of standard load testing, we evaluated the backend’s integration stability using Playwright’s API testing context. We monitored endpoint response times, which successfully identified critical bottlenecks, such as the complete 30-second hard timeouts on the external Job Sources API.
- **Evidence Collection:** We configured Playwright to automatically capture rich artifacts—including DOM trace files (`trace.zip`) and screenshots—the moment a test failed, ensuring our results are 100% reproducible for the developers.

2.2 Phase 2: Manual Exploratory & Security Testing

While the automation suite verified the “happy paths” and routing, we manually audited the platform using web proxies (Burp Suite), API clients (Postman, cURL), and browser Developer Tools. This phase focused entirely on breaking the application by stepping outside expected user behavior and bypassing the frontend interface entirely.

- **API Security & Authorization Testing:** Using Burp Suite and cURL, we intercepted HTTP traffic to test how the backend handles data without the frontend’s protection. We manipulated JSON payloads and UUIDs to test for object-level authorization (BOLA/IDOR) and Mass Assignment. This methodology was crucial in uncovering the platform’s most critical vulnerabilities, including unauthenticated account takeovers and mass credential harvesting.
- **Boundary Value & Fuzzing Analysis:** We tested system limits by pasting massive strings (50,000+ characters) into fields like Login, Register, and Search keywords to see how the client handled memory processing, resulting in browser freezes. We also fuzzed backend endpoints with malformed data and single-character inputs to intentionally trigger internal server errors and expose Java stack traces.
- **State & Session Testing:** We tested how the platform handles session lifecycles. For example, we logged into an account on one browser, changed the password on a second browser, and then verified if the first browser’s session was properly invalidated (it was not). We also confirmed that the logout button fails to properly redirect and invalidate users.
- **UI & Responsive Design Testing:** We systematically tested the frontend interface using the Brave and Chrome browsers, specifically looking for responsive design breaks on mobile viewports, unresolved placeholder text, and layout misalignments across the Demand and Supply forecasting modules.

By combining Playwright’s consistent automation with creative, adversarial manual testing, we were able to provide a general view of both the codebase’s structural integrity and its critical security flaws.

3 Key Findings

Our testing across the SKILLAB platform revealed 39 distinct defects, heavily concentrated in backend authorization, routing logic, and system stability. While the frontend provides a reasonable baseline for standard citizens, the underlying architecture implicitly trusts user input and fails to enforce critical security boundaries.

The findings are categorized into three primary areas of critical impact:

3.1 Critical Identity, Authorization, and Security Failures

The platform’s user identity management is severely compromised, allowing for unauthenticated data extraction and complete account takeovers.

- **BOLA and Data Exposure:** The `/user-management/user` endpoint suffers from a Critical Insecure Direct Object Reference (IDOR) and Broken Object Level Authorization (BOLA) vulnerability (SEC_003, IDOR_USER_API_001). Any authenticated user can supply an arbitrary UUID to extract other users’ Personally Identifiable Information (PII), active password reset tokens, and bcrypt password hashes.
- **Account Takeovers (ATO):** We discovered multiple persistent ATO vectors. The registration endpoint improperly executes database “upsert” operations, allowing attackers to overwrite existing user credentials (ATO_REGISTRATION_001). Furthermore, a Mass Assignment vulnerability allows attackers to inject future-dated password reset tokens (e.g., valid until 2099) into any user’s profile (ATO_MASS_ASSIGN_001), creating a silent backdoor.
- **Absence of Server-Side Validation:** The API completely fails to validate input on the backend (BROKEN_AUTH_VALIDATION_001), allowing the creation of accounts with empty strings or single-character credentials. The password reset execution flow (BROKEN_AUTH_PASS_RESET_001) only requires a 3-character string, bypassing all frontend complexity policies (REGISTER_002).
- **Broken Session Management:** Active sessions are not invalidated after a user changes their password (SEC_001), leaving accounts vulnerable if a device is compromised.

3.2 Role-Based Access Control (RBAC) and Routing Collapse

The platform’s enterprise and administrative dashboards are functionally inaccessible due to broken frontend routing and session handling.

- **Total Stakeholder Lockout:** When authenticated users with the Industry (RBAC-001), Policy (RBAC-003), or Admin (RBAC-004) roles attempt to access their respective dashboards, the application forcefully redirects them to the public homepage or login screen. This constitutes a total loss of system management and enterprise capabilities.
- **Broken Navigation Components:** While the Education role bypasses the violent redirect (RBAC-002), the sidebar component fails to render entirely, leaving the user stranded on the `/education/account` page with no way to navigate.

3.3 System Stability, API Integrity, and UI Inconsistencies

The system struggles to handle unexpected data, resulting in resource exhaustion, broken workflows, and verbose information leaks.

- **API Timeouts and Hangs:** The Job Sources endpoint (`/api/jobs/sources`) critically fails to respond, causing the request to hang and eventually time out after 30 seconds (API-002). Invalid search queries also cause the Turf Analysis request to hang in a “Pending” state indefinitely (SEARCH_002).

- **Information Disclosure:** Fuzzing the backend endpoints with malformed data or single-character inputs triggers unhandled exceptions instead of standardized HTTP errors (INFO_LEAK_ERROR_HANDLING_001, INFO_LEAK_ERROR_HANDLING_002). This leaks verbose Java stack traces and reveals internal routing mechanisms, such as the use of Google’s SMTP servers (ENUM_INFO_LEAK_PASS_RESET_001).
- **Frontend Resource Exhaustion:** Multiple input fields, including Login (LOGIN_001), Registration (REGISTER_001), and Search Keywords (SEARCH_003, SEARCH_004), lack character limits. Pasting exceptionally large strings (e.g., 50,000+ characters) completely freezes the browser tab.
- **UI/UX Misalignments:** The application exhibits several frontend inconsistencies, including layout shifts in the “My Skills” section (DESIGN_001), component misalignments across the Demand and Supply Analytics modules (DESIGN_002, DESIGN_003, DESIGN_004), and unresolved boilerplate placeholder text in the notification dropdown (UI_001).

4 Complete Defect Tracker

A list of all reported bugs is provided in Annex 1 at the end of this document.

4.1 Deliverables Included in the Testing Package

The accompanying testing artefact package contains all source code, scripts, and manual evidence generated during our campaign. The deliverables are structured as follows:

- **Automated Testing Suite (automated_testing/):** Contains full Playwright End-to-End (E2E) test specifications covering API endpoints, authentication flows, and RBAC functionality, complete with Page Object Models and centralized global setup configurations.
- **Performance & Load Tests (automated_testing/k6_testing/):** Includes k6 scripts configured to test both the local Docker instance and the live deployed portal, alongside saved JSON results and console execution logs.
- **Manual Testing Evidence (results_and_manual_testing/):** Houses the consolidated master bug report (SKILLAB_Master_Bug_Report.md), individual tester notes, and a directory of multimedia evidence containing 24 files of screen recordings (.mp4) and screenshots (.png) documenting functional bugs.
- **Security Testing Scripts (manual_security/):** Contains isolated reproduction steps (including cURL commands and raw Burp Suite requests) and explicit result screenshots for all identified vulnerabilities (e.g., ATO, IDOR, Mass Assignment).

4.2 How to Run the Tests

To reproduce our findings and execute the test suites, ensure your environment meets the minimum prerequisites: Docker (20.10+), Node.js (18+), npm (9+), and k6 (0.50+).

Step 1: Target Environment Setup

The automated functional tests are configured to run against a local Dockerized instance of the SKILLAB platform.

```
1 Bash
2 git clone https://github.com/skillab-project/SKILLAB-platform.git
3 cd SKILLAB-platform/all-in-one
4 docker compose up -d
```

Important: Wait approximately 1–2 minutes after execution for all containers (frontend at `localhost:3000`, backend at `localhost:8081`) to fully initialize before proceeding.

Step 2: Executing Automated UI/E2E Tests (Playwright)

Navigate to the testing directory to install dependencies and execute the full suite. The runner will automatically handle authenticating all 5 user roles via the global setup.

```
1 Bash
2 cd automated_testing
3 npm install
4 npx playwright install
5 npx playwright test
```

To view the interactive HTML report—which includes DOM traces, screenshots, and videos of any failed tests—run:

```
1 Bash
2 npx playwright show-report
```

Step 3: Executing Performance Tests (k6)

To evaluate the platform’s load handling and verify our response time findings, navigate to the k6 directory and run the local load test:

```
1 Bash
2 cd automated_testing/k6_testing
3 k6 run login_load_test.js
```

(Note: To run the load test against the live deployed instance instead, execute `k6 run deployed_login_load_test.js`).

5 Recommendations

To successfully transition the SKILLAB platform from its current state to a secure, production-ready release, the development team must shift its focus from frontend features to backend structural integrity. Our assessment indicates that the API currently operates on an implicit trust model—relying too heavily on the client side to validate data and enforce rules.

To address the root causes of the vulnerabilities discovered, we strongly recommend implementing the following architectural changes:

5.1 Enforce Zero-Trust API Architecture and Strict Authorization

The most critical risk to the platform is the backend’s failure to verify user permissions at the object level.

- **Implement Object-Level Checks:** Every API endpoint handling user data (especially `/user-management/user`) must explicitly verify that the authenticated user’s session ID matches the UUID being requested or modified.
- **Utilize Data Transfer Objects (DTOs):** The backend currently returns entire database entities to the frontend. Implement strict DTOs to strip sensitive backend fields—such as bcrypt password hashes and `passResetCode`—from all API responses.
- **Block Mass Assignment:** The registration and update endpoints must reject unexpected fields. Explicitly define which fields can be updated via API to prevent attackers from injecting future-dated password reset tokens or overwriting existing user IDs (the “upsert” flaw).
- **Mandate Server-Side Validation:** Never trust the client. Password complexity, email formatting, and input length must be strictly enforced on the server. Bypassing the UI should not allow a user to set a 3-character password.

5.2 Repair Enterprise Routing Middleware and Session Management

The platform cannot be adopted by organizational stakeholders if they cannot access their workspaces.

- **Audit RBAC Routing Logic:** The frontend routing middleware must be reviewed to identify why Admin, Industry, and Policy roles are encountering violent redirects to the root URL. Ensure that the role claims within the JWT are correctly mapped to the respective dashboard components.
- **Enforce Session Invalidation:** Implement robust session management that immediately destroys all active JWTs and session states across all devices when a user changes their password or clicks logout.

5.3 Implement Global Exception Handling and Rate Limiting

The system currently fails ungracefully, which degrades the user experience and provides attackers with a roadmap of the backend infrastructure.

- **Standardize Error Responses:** Implement a global exception handler across the backend controllers. Catch raw Java exceptions (like database relation errors or SMTP failures) and return standardized, sanitized HTTP 400 or 500 JSON responses. Internal stack traces must never reach the client.
- **Enforce Rate Limiting:** Apply strict rate limiting to sensitive endpoints, particularly the password reset flow, to prevent automated email bombing and safeguard the platform’s email domain reputation.

5.4 Sanitize Inputs and Stabilize External Integrations

Resource exhaustion and hanging requests currently break the application’s core functionality.

- **Apply Hard Input Constraints:** Implement strict character limits (e.g., 255 characters for standard fields) on both the HTML forms and the backend schema. This will immediately resolve the browser freezing issues caused by large payload injections.
- **Implement Circuit Breakers:** For external integrations like the Job Sources API (`skillab-tracker.csd.auth.gr`), implement a circuit breaker pattern or strict timeout limits (e.g., 5 seconds) so that if the external service goes down, the SKILLAB frontend can display a graceful “Service Unavailable” message instead of hanging indefinitely.

Annex 1: Security and QA Findings

The file `results_and_manual_testing/SKILLAB_Master_Bug_Report.md` from the delivered materials contains the full details, including their severity, affected components, and step-by-step reproduction instructions.

#	ID	Title	Category	Severity	Priority
1	SEC_003	BOLA in /user-management/user/ Endpoint Exposes Sensitive User Data (Password Hashes and Reset Codes)	Security	Critical	Critical
2	IDOR_USER_API_001	Critical IDOR Exposes PII, Password Hashes, and Authentication Secrets	Security	Critical	Critical
3	ATO_REGISTRATION_001	Account Takeover via Improper Upsert Handling on Registration Endpoint	Security	Critical	Critical
4	ATO_MASS_ASSIGN_001	Persistent Account Takeover via Mass Assignment of Password Reset Tokens	Security	Critical	Critical
5	ATO_TOKEN_THEFT_001	Complete Account Takeover via IDOR and Password Reset Token Theft	Security	Critical	Critical
6	RBAC-004	Admin Role Locked Out of Admin Panel	RBAC / Security	Critical	Critical
7	RBAC-003	Policy Role Redirected to Public Homepage	RBAC / Security	Critical	High
8	UI-002	Improper Redirect on User Logout	UI / Auth Flow	Critical	High
9	REGISTER_002	Register form lacks password policy validation (length and complexity)	Security	High	High
10	SEC_001	Session remains active after password change	Security	High	High
11	SEC_002	Improper input handling leads to Internal Server Error (500)/Potential XSS	Security	High	High
12	BROKEN_AUTH_VALIDATION_001	Complete Lack of Server-Side Input Validation on Registration Endpoint	Security	High	High

#	ID	Title	Category	Severity	Priority
13	BROKEN_AUTH_PASS_RESET_001	Weak Password Policy Enforcement on Password Reset Execution	Security	High	High
14	API-002	Critical Timeout on Job Sources Endpoint (GET /api/jobs/sources)	API	High	High
15	UI-001	Inaccessible Forecasting Link in Citizen Sidebar	UI / Navigation	High	High
16	RBAC-001	Industry Role Completely Locked Out of Dashboard	RBAC / Security	Critical	Medium
17	RBAC-002	Education Role Missing Sidebar Navigation	RBAC / Security	Critical	Medium
18	FORGOT_PW_001	Forgot Password feature is vulnerable to user enumeration	Security	Medium	Medium
19	LOGIN_001	Login form allows unlimited character input, causing page freeze	UI/UX	Medium	Medium
20	LOGIN_003	Login form accepts empty/blank credentials	Functional	Medium	Medium
21	REGISTER_001	Register form allows unlimited character input, causing page freeze, and the form to break	UI/UX	Medium	Medium
22	SEARCH_001	Error for invalid search and loading state persists indefinitely after failed search validation	Functional	Medium	Medium
23	SEARCH_002	Search request hangs in Pending state when entering invalid keywords	Integration	Medium	Medium
24	SEARCH_003	Keywords field allows unlimited character input, causing page freeze	UI/UX	Medium	Medium
25	SEARCH_004	Keywords field from Supply/Analytics/Co-occurrence allows unlimited character input, causing page freeze	UI/UX	Medium	Medium

#	ID	Title	Category	Severity	Priority
26	INFO_LEAK_ERROR_HANDLING_001	Information Disclosure via Improper Error Handling on User Management Endpoint (GET/POST)	Security	Medium	Medium
27	ENUM_INFO_LEAK_PASS_RESET_001	User Enumeration and Information Disclosure via Improper Exception Handling on Password Reset	Security	Medium	Medium
28	RATE_LIMIT_PASS_RESET_001	Lack of Rate Limiting on Password Reset Enables Email Bombing	Security	Medium	Medium
29	INFO_LEAK_ERROR_HANDLING_002	Information Disclosure via Improper Error Handling on Password Reset Endpoint	Security	Medium	Medium
30	API-001	Empty Response Body on Failed Authentication (POST /login)	API	Medium	Medium
31	UX_002	Mobile layout breaks when accessing navigation menus	UI/UX	Low	Medium
32	LOGIN_002	Failed login attempt returns HTTP 403 Forbidden instead of 401 Unauthorized	Integration	Low	Low
33	UX_001	Missing auto-login flow after successful registration	UI/UX	Low	Low
34	UX_003	Generic and misleading error message displayed instead of specific input validation error	UI/UX	Low	Low
35	DESIGN_001	Layout shift in My Skills section	UI/UX	Low	Low
36	DESIGN_002	Layout misalignment in Demand and Supply, Forecasting/Ageing sections	UI/UX	Low	Low
37	DESIGN_003	Layout misalignment in Deman/Analytics, Turf Analysis form	UI/UX	Low	Low
38	DESIGN_004	Layout misalignment in Supply/Analytics, Turf Analysis form	UI/UX	Low	Low

#	ID	Title	Category	Severity	Priority
39	UI_001	Unresolved placeholder text in notification dropdown	UI/UX	Low	Low